

Hey Trale — Here's How This All Works

You helped build a real website that real people are using. That's not a small thing. This doc is your guide to understanding what we built, how the pieces fit together, and how to start making it your own.

The Big Picture

Here's the one-sentence version: **someone visits a web address, fills out a form, clicks submit, and their picks are saved — and after the tournament starts, the site automatically scores everyone and shows a leaderboard.**

That sounds simple, but there are about five different systems involved every time it happens. Before we break those down, it helps to understand one fundamental thing about how websites work:

The website you see in your browser is not where the work happens.

Your browser (Chrome, Safari, whatever) is just a display screen. When you click Submit, your browser sends a message over the internet to a computer somewhere else — called a **server** — and that server does the actual work. In our case, that server is running our Python code 24 hours a day, waiting for someone to show up.

We didn't have to buy a server or set one up ourselves. Streamlit Community Cloud does that for free. All we did was push our code to GitHub, tell Streamlit "here's our code," and Streamlit took care of the rest.

What Actually Happens, Step by Step

When someone submits their bracket:

- 1. They open the website on their phone or laptop.** Their browser sends a request to Streamlit's servers, which run our `app.py` file. `app.py` builds the page — the bracket form, the orange buttons, the progress bar — and sends it back to their browser to display.
- 2. They fill in their picks and click Submit.** Their browser packages up all 63 picks plus their name and sends it back to the server, like handing in a completed test.
- 3. Our code receives those picks and saves them.** `app.py` passes the picks to `sheets.py`, which connects to Google Sheets and writes a new row — one column per game. From that moment, their picks live in a Google spreadsheet that we can see and that the scoring system can read.

The person's browser never touched Google Sheets directly. Everything went through our Python code on Streamlit's server first. That's how almost all web apps work: the browser talks to a server, and the server talks to everything else.

When someone views the leaderboard:

- 1. They open the website after the deadline.** Same as before — their browser asks Streamlit's server to run our code. This time, `app.py` sees that the deadline has passed and decides to show the leaderboard instead of

the form.

2. Our code goes and gets everyone's picks. `sheets.py` reads every row from Google Sheets — every person's name and all 63 of their picks. This happens fresh every time someone loads the page.

3. Our code asks ESPN for game scores. `espn_api.py` sends a request to an ESPN web address that returns a big list of completed game results — teams, scores, everything. ESPN built this for their own apps, but it's publicly accessible, so we use it too. (This is called an **API** — more on that in the vocabulary section.)

4. Our code does the math. `scoring.py` compares each person's picks against the actual results and adds up their points: 10 for each correct pick, plus upset bonus, plus margin bonus.

5. The results get sent back to the browser. `app.py` builds the leaderboard page — sorted by score, with medals for the top three — and sends it to the browser to display.

The whole thing happens in about one second, every time someone loads the page.

The Files, Plain English

`config.py` — The Settings Panel

This is like the control room. It stores important numbers and settings that other files use. If you ever needed to change the submission deadline or the point values, this is where you'd do it — one change here affects the whole app.

```
SUBMISSION_DEADLINE = CT.localize(datetime(2026, 3, 19, 11, 0, 0))
POINTS_PER_WIN = 10
UPSET_BONUS_PTS = 4
```

`bracket_data.py` — The Bracket Blueprint

This file describes the entire tournament structure. Every team, every seed, and every game — including which games feed into which later games. It's like a map of the whole bracket.

The coolest part: each game knows where its winner goes next. Game g1 (Duke vs Siena) says its winner goes to g33 (Round of 32). Game g33's winner goes to g49 (Sweet 16). And so on, all the way to g63 (Championship). The bracket is just one long chain of games.

`espn_api.py` — The Score Fetcher

ESPN has a secret (unofficial) web address that returns all current basketball scores in a format computers can read called **JSON** — basically a big organized list. This file sends a request to that address, reads the response, and translates ESPN's team names into our bracket's team names.

This is called an **API** (Application Programming Interface). You've probably used APIs without knowing it — whenever your weather app shows the forecast, it's talking to a weather API. We're doing the same thing with sports scores.

`scoring.py` — The Referee

This file takes two things — what everyone picked, and what actually happened — and calculates the points. It checks three things for every completed game:

1. Did you pick the right winner? (+10)
2. Was it an upset? (+4)
3. How big was the margin? (+half)

`sheets.py` — The Filing Cabinet

Everything that needs to be saved (picks) or retrieved (picks, for scoring) goes through this file. It talks to Google Sheets using a "service account" — think of that as a robot Google account that only has permission to read and write one specific spreadsheet.

`app.py` — The Whole Website

This is the biggest file and the one you'll interact with most. It builds everything you see on screen. Streamlit is unusual compared to most web frameworks — instead of writing HTML, you write Python and Streamlit turns it into a webpage automatically.

For example, this one line:

```
st.title("Trale's 2026 NCAA Bracket Pool")
```

...creates the big heading at the top of the page. That's it. One line.

```
st.progress(pct)
```

...draws the orange progress bar showing how many picks you've made.

```
st.tabs(["🏀 East", "🏀 South", ...])
```

...creates the tab strip with all four regions.

Try These Changes

These are safe experiments. Each one is a small change you can make and then push to GitHub to see it live on the website within about 60 seconds.

Experiment 1 — Change the Title

Open `app.py`. Find this line (around line 100):

```
st.title("🏀 Trale's 2026 NCAA Bracket Pool")
```

Change it to whatever you want:

```
st.title("🏀 Trale's Bracket Domination 2026")
```

Save the file. Then push to GitHub (see the section below). Reload the site.

Experiment 2 — Change the Orange Color

Open `app.py`. Near the top, find the CSS section with this line:

```
--orange:      #E8651A;
```

That `#E8651A` is a **hex color code** — a way computers describe colors. Try swapping it for one of these:

Color	Code
Bright red	<code>#E83A1A</code>
Hot pink	<code>#E81A8C</code>
Purple	<code>#7B1AE8</code>
Lime green	<code>#39E81A</code>

Save, push, reload. The buttons, progress bar, score highlights, and accents will all change to your new color.

Experiment 3 — Add a Fun Message

In `app.py`, find the leaderboard section. Look for this line:

```
st.markdown("Submissions are closed. Good luck everyone! 🏀")
```

Change the message to something more fun:

```
st.markdown("Submissions are closed. May the best bracket win. 🏆💧")
```

Experiment 4 — Change the Scoring (careful!)

Open `config.py`. Find:

```
POINTS_PER_WIN = 10
UPSET_BONUS_PTS = 4
```

Try changing the upset bonus to 8 and see how it reshuffles the leaderboard. Change it back when you're done — this affects real scores!

Experiment 5 — Add an Emoji to the Countdown

In `app.py`, find the countdown section:

```
f'🕒 Submissions close in <strong>{hours}h {minutes}m</strong>'
```

Add more emojis or change the text however you want.

How to Push Changes to GitHub (and See Them Live)

When you save a file on your computer, the website doesn't know yet. You have to **push** your changes to GitHub, which then tells Streamlit to update the site. Here's how:

Option A: GitHub Desktop (easiest)

1. Open GitHub Desktop
2. You'll see your changed files listed on the left
3. At the bottom left, type a short description of what you changed (e.g., "Changed the title")
4. Click **Commit to main**
5. Click **Push origin** at the top
6. Go to your Streamlit app URL and wait about 60 seconds — it'll reload automatically

Option B: Terminal (more like a real developer)

Open a terminal in the project folder and type these three commands:

```
git add .
git commit -m "describe what you changed here"
git push
```

That's it. Three commands every time.

Vocabulary Cheat Sheet

Word	What it means
Python	The programming language we used. Reads almost like English.

Word	What it means
Streamlit	A Python tool that turns code into websites automatically
API	A way for two programs to talk to each other over the internet
JSON	A format for organizing data that computers can read easily
Git	A system that tracks every change ever made to your code
GitHub	A website that stores your code and its history
Commit	Saving a snapshot of your code with a description
Push	Sending your saved snapshot to GitHub
Deploy	Making your code available on a real server so others can use it
Session state	How Streamlit remembers things while you're on the page
CSS	A language for styling websites — colors, fonts, spacing
Hex color	A 6-character code like #E8651A that describes a color precisely
Cache	Storing a result so you don't have to recalculate it every time
Service account	A robot Google account used by programs instead of humans

What You Built, For Real

Most people your age (and honestly most adults) don't know how to take an idea and turn it into a working website that other people can use. You do now — or at least you've seen it happen and you understand the pieces.

The skills here — Python, APIs, databases, version control — are the same ones used by engineers at every tech company. The tools we used are just smaller and free versions of the same concepts.

If something here confuses you, ask. If something interests you, dig in. The best way to learn this stuff is to break things and figure out why they broke.

Built March 2026 by Dad + Trale using Claude Code.